Running MapReduce WordCount Program in Python

Before running the below commands make sure that you have :

- Switched to the **hadoop user**.
- Navigated to the hadoop **Sbin** directory.
- Started the **Hadoop Distributed File System (HDFS).**
- Started the **YARN Resource Manager**.

If not refer to the below image to complete the above tasks :

```
hdoop@Siddhartha-Shakya: ~
siddhu@Siddhartha-Shakya:~$ jps
441 Jps
siddhu@Siddhartha-Shakya:~$ su - hdoop
Password:
hdoop@Siddhartha-Shakya:~$ cd hadoop-3.2.4/sbin/
hdoop@Siddhartha-Shakya:~/hadoop-3.2.4/sbin$ ./start-dfs.sh
Starting namenodes on [localhost]
Starting datanodes
Starting secondary namenodes [Siddhartha-Shakya]
hdoop@Siddhartha-Shakya:~/hadoop-3.2.4/sbin$ ./start-yarn.sh
Starting resourcemanager
Starting nodemanagers
hdoop@Siddhartha-Shakya:~/hadoop-3.2.4/sbin$ jps
2020 Jps
1477 ResourceManager
1626 NodeManager
781 NameNode
910 DataNode
1183 SecondaryNameNode
hdoop@Siddhartha-Shakya:~/hadoop-3.2.4/sbin$
```

If **jps (Java Processing Status)** lists all the above **running java processes**, then you are **good to continue** with the below commands**.**

1. Python comes pre-installed in newer versions of Ubuntu. However, if you want to check which version is installed on your machine:

For python version check :

```
~$ python3 --version
```

2. If the python is not installed then you can install the python with the following code:

```
~$ sudo apt install python3
```

3. Create **input_word** folder and **insert a text file** in the **HDFS**.

```
~$ hdfs dfs -mkdir /input_word

~$ hdfs dfs -put hadoop-3.2.4/LICENSE.txt /input_word/
```

4. Verify the creation of the **directory** and the **file**.

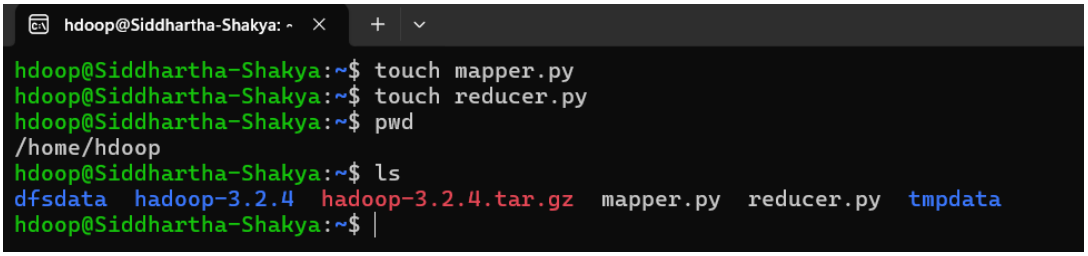
```
~$ hdfs dfs -ls /

~$ hdfs dfs -ls /input_word/

~$
```

5. Now we have to create a mapper and reducer. To create a mapper class first we have to create an empty file and add the code related.

6. Type "**touch mapper.py**" and press enter. Similarly for reducer "**touch reducer.py**" and press enter. You will have two files located in your **/home/hdoop** directory.



7. Open the mapper.py typing "**nano mapper.py**" and add the following code.

```
~$ nano mapper.py
```

Copy paste the below code :

```python
#!/usr/bin/env python3
"""mapper.py"""

import sys

# input comes from STDIN (standard input)
for line in sys.stdin:
    # remove leading and trailing whitespace
    line = line.strip()
    # split the line into words
    words = line.split()
    # increase counters
    for word in words:
        # write the results to STDOUT (standard output);
        # what we output here will be the input for the
        # Reduce step, i.e. the input for reducer.py
        #
        # tab-delimited; the trivial word count is 1
        print("%s\t%s" % (word, 1))
```

8. Now open the reducer file by typing "**reducer.py**" and press enter. add the following code in reducer.py.

```
~$ nano reducer.py
```

Copy paste the below code :

```python
#!/usr/bin/env python3
"""reducer.py"""
from operator import itemgetter
import sys

current_word = None
```

```
current_count = 0
word = None

# input comes from STDIN
for line in sys.stdin:
    # remove leading and trailing whitespace
    line = line.strip()
    # parse the input we got from mapper.py
    word, count = line.split("\t", 1)
    # convert count (currently a string) to int
    try:
        count = int(count)
    except ValueError:
        # count was not a number, so silently
        # ignore/discard this line
        continue

    # this IF-switch only works because Hadoop sorts map output
    # by key (here: word) before it is passed to the reducer
    if current_word == word:
        current_count += count
    else:
        if current_word is not None:
            # write result to STDOUT
            print("%s\t%s" % (current_word, current_count))
        current_count = count
        current_word = word

# do not forget to output the last word if needed!
if current_word == word:
    print("%s\t%s" % (current_word, current_count))
```

9. Now we have to give permission for the mapper and reducer. Type the following commands.

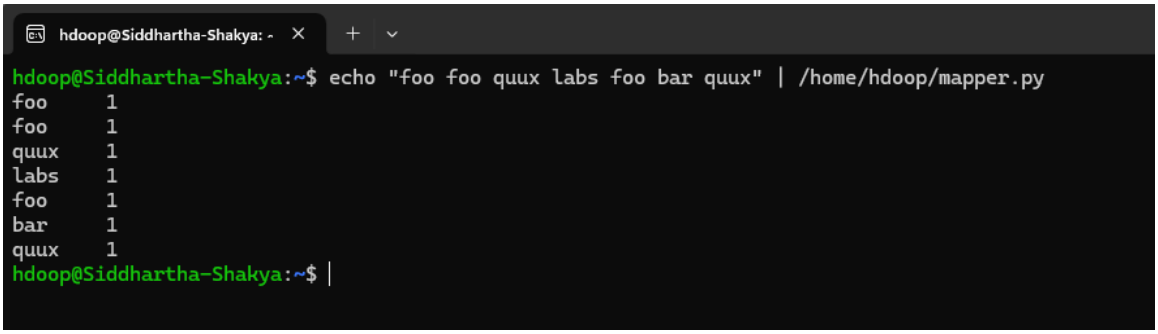
```
~$ chmod +x /home/hdoop/mapper.py

~$ chmod +x /home/hdoop/reducer.py
```

Testing the mapper.py and reducer.py

10. To test the mapper type the following code and enter.

```
~$ echo "foo foo quux labs foo bar quux" | /home/hdoop/mapper.py
```



11. Now test the mapper and reducer code together

```
~$ echo "foo foo quux labs foo bar quux" | /home/hdoop/mapper.py | sort -k1,1
| /home/hdoop/reducer.py
```

Running the Mapper.py and reducer.py in HDFS

12. Now to execute the code run the following code.

```
~$ hadoop jar /home/hdoop/hadoop-3.2.4/share/hadoop/tools/lib/hadoop-streaming-3.2.4.jar \
 -mapper /home/hdoop/mapper.py \
 -reducer /home/hdoop/reducer.py \
 -input /input_word/LICENSE.txt \
```

```
-output /output_word
```

13. if you have output folder then you have to delete that

<u>Note: It is just the syntax. Please enter proper credentials as your system has been configured.</u>

```
hdoop@Siddhartha-Shakya:~$ hadoop jar /home/hdoop/hadoop-3.2.4/share/hadoop/tools/lib/hadoop-streaming-3.2.4.jar \
  -mapper /home/hdoop/mapper.py \
  -reducer /home/hdoop/reducer.py \
  -input /input_word/LICENSE.txt \
  -output /output_word
```

```
                Virtual memory (bytes) snapshot=7705022464
                Total committed heap usage (bytes)=733478912
                Peak Map Physical memory (bytes)=328499200
                Peak Map Virtual memory (bytes)=2570252288
                Peak Reduce Physical memory (bytes)=188260352
                Peak Reduce Virtual memory (bytes)=2570457088
        Shuffle Errors
                BAD_ID=0
                CONNECTION=0
                IO_ERROR=0
                WRONG_LENGTH=0
                WRONG_MAP=0
                WRONG_REDUCE=0
        File Input Format Counters
                Bytes Read=154667
        File Output Format Counters
                Bytes Written=34238
2025-04-03 20:37:02,023 INFO streaming.StreamJob: Output directory: /output_word
hdoop@Siddhartha-Shakya:~$
```

14. Now you will see the output folder and the output file. **output_word** in my case as I have given that specific name while executing map reduce code.

```
hdoop@Siddhartha-Shakya: ~
hdoop@Siddhartha-Shakya:~$ hdfs dfs -ls /
Found 3 items
drwxr-xr-x   - hdoop supergroup          0 2025-04-03 20:23 /input_word
drwxr-xr-x   - hdoop supergroup          0 2025-04-03 20:37 /output_word
drwx------   - hdoop supergroup          0 2025-04-03 19:59 /tmp
hdoop@Siddhartha-Shakya:~$ hdfs dfs -ls /output_word
Found 2 items
-rw-r--r--   1 hdoop supergroup          0 2025-04-03 20:37 /output_word/_SUCCESS
-rw-r--r--   1 hdoop supergroup      34238 2025-04-03 20:37 /output_word/part-00000
hdoop@Siddhartha-Shakya:~$
```

15. Now you can test if the map reduce program in the python worked correctly or not by executing the following command.

```
~$ hdfs dfs -cat /output_word/part-00000
```

```
hdoop@Siddhartha-Shakya: ~
within  21
without 74
words;  1
work    36
work,   8
work.   3
work;   2
works   10
works,  4
world-wide,     5
worldwide,      6
would   5
writing 5
writing,        5
writing;        1
written 21
year    1
you     17
your    4
§       1
“AS     1
“Contribution”  1
“Contributor    2
“Contributor”   2
“Covered        2
“Executable     1
“Executable”    1
“Incompatible   3
“Initial        1
“Larger 2
“Licensable”    2
“License”       2
“Modifications” 2
“Original       1
“Participant”)  1
“Patent 2
“Secondary      1
“Source 2
“Your”) 2
“You”   4
“as     1
“commercial     3
“control”       2
hdoop@Siddhartha-Shakya:~$
```

!! WELL DONE !!

16. References

- [Writing An Hadoop MapReduce Program In Python]